

PC Algorithm From Scratch

End-to-end build guide: dataset import → causal skeleton → edge orientation → structure-recovery evaluation, on the Tennessee Eastman Process chemical plant dataset.

A note before you start: the PC algorithm outputs a **causal graph (CPDAG)**, not class labels. So there is no scikit-learn-style `classification_report` at the end. The equivalent evaluation step here is **edge-level precision / recall / F1 and Structural Hamming Distance (SHD)** against the ground-truth causal graph — that's what Step 9 builds.

Step 0 — Environment setup

Install the packages you'll need. `numpy/scipy` handle the linear algebra and statistics behind the CI test; `pandas` loads the dataset; `networkx` is just used for graph bookkeeping and drawing (not for the algorithm's logic itself — you're writing that).

```
pip install numpy scipy pandas networkx matplotlib --break-system-packages
```

Step 1 — Import the dataset

Download the Tennessee Eastman Process (TEP) data + ground-truth causal graph from the CIPCaD-Bench repository: github.com/giovanniMen/CPCaD-Bench. Load the process variable CSV into a pandas DataFrame, and load the ground-truth adjacency (from their provided graph file) separately — you will NOT touch the ground truth until Step 9.

```
import pandas as pd
import numpy as np

def load_dataset(csv_path):
    """
    Loads the TEP process data.
    Rows = timestamps, Columns = process variables (e.g. XMEAS(1)...XMEAS(33)).
    Returns a numpy array (n_samples x n_variables) and the variable names.
    """
    df = pd.read_csv(csv_path)
    var_names = list(df.columns)
    data = df.to_numpy(dtype=float)
    return data, var_names

def load_ground_truth(edge_list_path, var_names):
    """
    Loads the ground-truth causal graph as an adjacency matrix, for
    evaluation ONLY (Step 9). Format: CSV of (source, target) edges.
    """
    n = len(var_names)
    idx = {v: i for i, v in enumerate(var_names)}
    gt = np.zeros((n, n), dtype=int)
    edges = pd.read_csv(edge_list_path)
    for _, row in edges.iterrows():
        i, j = idx[row["source"]], idx[row["target"]]
        gt[i, j] = 1 # directed edge i -> j
    return gt
```

Step 2 — Preprocess (standardize)

Standardizing each column to zero mean / unit variance does not change the causal structure (partial correlation is affine-invariant) but it keeps the correlation-matrix inversion in Step 4 numerically stable, which matters a lot with 30+ correlated plant variables.

```
def standardize(data):
    """
    z_ij = (x_ij - mean_j) / std_j    for each column j
    Keeps partial-correlation computations numerically stable.
    """
    mean = data.mean(axis=0)
    std = data.std(axis=0)
    std[std == 0] = 1.0 # guard against constant columns
    return (data - mean) / std
```

Step 3 — Conditional independence test

This is the statistical core. For variables X , Y and conditioning set Z , we test $X \perp Y \mid Z$ using partial correlation derived from the precision matrix (inverse covariance), converted to a Fisher z-statistic.

Mathematically: given the correlation matrix restricted to $\{X, Y\} \cup Z$, compute $P = \Sigma^{-1}$ (the precision matrix). Then:

```
rho_XY.Z = - P[X,Y] / sqrt(P[X,X] * P[Y,Y])

z = 0.5 * ln( (1 + rho_XY.Z) / (1 - rho_XY.Z) )
statistic = sqrt(n - |Z| - 3) * |z|
```

Decision: if `statistic < Phi_inverse(1 - alpha/2)` -> independent (remove edge)
from `scipy.stats` import `norm`

```
def partial_corr_test(data, x, y, z, alpha=0.05):
    """
    Tests  $X \perp Y \mid Z$  using partial correlation (Fisher z-test).
    x, y: column indices (ints)
    z: list of column indices (conditioning set)
    Returns True if independent (edge should be removed), False otherwise.
    """
    n = data.shape[0]
    cols = [x, y] + list(z)
    sub_corr = np.corrcoef(data[:, cols], rowvar=False)

    try:
        precision = np.linalg.inv(sub_corr)
    except np.linalg.LinAlgError:
        precision = np.linalg.pinv(sub_corr) # fallback if singular

    p_xy = precision[0, 1]
    p_xx = precision[0, 0]
    p_yy = precision[1, 1]

    denom = np.sqrt(p_xx * p_yy)
    if denom == 0:
        return True # degenerate case, treat as independent

    rho = -p_xy / denom
    rho = np.clip(rho, -0.999999, 0.999999) # avoid log(0)

    z_stat = 0.5 * np.log((1 + rho) / (1 - rho))
    test_stat = np.sqrt(max(n - len(z) - 3, 1)) * abs(z_stat)
```

```
threshold = norm.ppf(1 - alpha / 2)
return test_stat < threshold # True -> independent -> remove edge
```

If the plant's relationships are nonlinear (very likely for reactor kinetics), swap this function for a kernel-based (KCI) or rank-based test later — the rest of the pipeline doesn't need to change, since every other function just calls this one as a black box.

Step 4 — Adjacency helper

A small graph-only utility (no statistics) used repeatedly during skeleton discovery to fetch a node's current neighbors.

```
def get_adjacent(adj_matrix, x):
    """
    Returns list of node indices currently adjacent to x.
    adj_matrix: symmetric 0/1 matrix representing the undirected skeleton.
    """
    return [i for i in range(adj_matrix.shape[0]) if adj_matrix[x, i] == 1 and i != x]
```

Step 5 — Skeleton discovery

Start from the complete graph. For growing conditioning-set size $k = 0, 1, 2, \dots$, test every remaining edge (X, Y) against every subset $Z \subseteq \text{Adj}(X) \setminus \{Y\}$ of size k . Remove the edge on the first Z that makes X and Y conditionally independent, and store that Z as the separating set — you need it again in Step 6.

```
from itertools import combinations

def skeleton_discovery(data, alpha=0.05):
    """
    Builds the undirected skeleton via iterative CI testing.
    Returns: adjacency matrix (0/1, symmetric) and a sepset dictionary
            sepset[(x,y)] = conditioning set that separated x and y.
    """
    n_vars = data.shape[1]
    adj_matrix = np.ones((n_vars, n_vars), dtype=int)
    np.fill_diagonal(adj_matrix, 0)
    sepset = {}

    ell = 0
    while True:
        edges_before = adj_matrix.sum()
        made_change = False

        for x in range(n_vars):
            for y in range(n_vars):
                if x >= y or adj_matrix[x, y] == 0:
                    continue

                neighbors = [v for v in get_adjacent(adj_matrix, x) if v != y]
                if len(neighbors) < ell:
                    continue

                for z_subset in combinations(neighbors, ell):
                    if partial_corr_test(data, x, y, list(z_subset), alpha):
                        adj_matrix[x, y] = 0
                        adj_matrix[y, x] = 0
                        sepset[(x, y)] = list(z_subset)
                        sepset[(y, x)] = list(z_subset)
                        made_change = True
                        break # stop checking more subsets for this edge

                ell += 1
            max_degree = max(len(get_adjacent(adj_matrix, x)) for x in range(n_vars))
            if ell > max_degree:
                break

        return adj_matrix, sepset
```

Step 6 — Orient v-structures (colliders)

For every unshielded triple $X - Z - Y$ (X, Y not adjacent), check whether Z was needed to separate X and Y . If Z is absent from $\text{Sepset}(X, Y)$, Z must be a collider: orient as $X \rightarrow Z \leftarrow Y$. This relies on the fact that conditioning on a true collider opens a path rather than closing it — so if the CI test found independence without conditioning on Z , Z can't be a fork or chain on that path.

```
def orient_v_structures(adj_matrix, sepset):
    """
    Detects colliders and orients them.
    Returns a directed adjacency matrix `direction` where
    direction[i, j] = 1 means edge oriented i -> j (0 = no arrowhead / still undirected).
    Undirected edges are represented as direction[i,j] == direction[j,i] == 0
    while adj_matrix[i,j] == 1 (i.e. edge exists, orientation pending).
    """
    n_vars = adj_matrix.shape[0]
    direction = np.zeros((n_vars, n_vars), dtype=int)

    for z in range(n_vars):
        neighbors_z = get_adjacent(adj_matrix, z)
        for x, y in combinations(neighbors_z, 2):
            if adj_matrix[x, y] == 1:
                continue # x, y adjacent -> shielded triple, skip

            sep = sepset.get((x, y), [])
            if z not in sep:
                # z is a collider: orient x -> z <- y
                direction[x, z] = 1
                direction[y, z] = 1

    return direction
```

Step 7 — Meek's rules (propagate orientation)

Repeatedly apply four local rules until nothing changes, to orient as many of the remaining undirected edges as the data logically permits — without creating a new collider (which would contradict Step 6) or a directed cycle.

```
def apply_meek_rules(adj_matrix, direction):
    """
    R1: x -> z, z - y, x,y non-adjacent          => orient z -> y
    R2: x -> z -> y, x - y                        => orient x -> y
    R3: x - z, x-w1->z, x-w2->z, w1,w2 non-adjacent => orient x -> z
    R4: acyclicity-preserving orientation rule
    Runs to a fixed point.
    """
    n_vars = adj_matrix.shape[0]

    def is_directed(i, j):
        return direction[i, j] == 1 and direction[j, i] == 0

    def is_undirected(i, j):
        return adj_matrix[i, j] == 1 and direction[i, j] == 0 and direction[j, i] == 0

    def adjacent(i, j):
        return adj_matrix[i, j] == 1

    changed = True
    while changed:
        changed = False
        # R1
```

```

for x in range(n_vars):
    for z in range(n_vars):
        if not is_directed(x, z):
            continue
        for y in range(n_vars):
            if is_undirected(z, y) and not adjacent(x, y):
                direction[z, y] = 1
                changed = True

# R2
for x in range(n_vars):
    for y in range(n_vars):
        if not is_undirected(x, y):
            continue
        for z in range(n_vars):
            if is_directed(x, z) and is_directed(z, y):
                direction[x, y] = 1
                changed = True

# R3
for x in range(n_vars):
    for z in range(n_vars):
        if not is_undirected(x, z):
            continue
        w_candidates = [w for w in range(n_vars)
                        if is_undirected(x, w) and is_directed(w, z)]
        for w1, w2 in combinations(w_candidates, 2):
            if not adjacent(w1, w2):
                direction[x, z] = 1
                changed = True

return direction

```

Step 8 — Orchestrator: pc_algorithm()

Ties every function together into the final callable. This is the only function you (or Claude in VS Code) needs to call from the outside.

```
def pc_algorithm(data, alpha=0.05):
    """
    Full PC algorithm.
    data: (n_samples, n_vars) numpy array, already standardized.
    Returns: adj_matrix (skeleton, 0/1 symmetric), direction (oriented edges)
    """
    adj_matrix, sepset = skeleton_discovery(data, alpha)
    direction = orient_v_structures(adj_matrix, sepset)
    direction = apply_meek_rules(adj_matrix, direction)
    return adj_matrix, direction
```

Step 9 — Run it end-to-end

```
data, var_names = load_dataset("tep_data.csv")
data = standardize(data)

adj_matrix, direction = pc_algorithm(data, alpha=0.05)

# Inspect discovered directed edges
n_vars = len(var_names)
for i in range(n_vars):
    for j in range(n_vars):
        if direction[i, j] == 1:
            print(f"{var_names[i]} -> {var_names[j]}")
```

Step 10 — Evaluate against ground truth

Now, and only now, bring in the ground-truth graph from Step 1. Compute edge-level precision, recall, F1, and Structural Hamming Distance (SHD) — the causal-discovery analog of a classification report.

```
def evaluate_against_ground_truth(direction, ground_truth):
    """
    direction, ground_truth: n x n binary matrices, entry [i,j]=1 means edge i -> j.
    Precision = TP / (TP + FP)
    Recall    = TP / (TP + FN)
    F1        = 2 * P * R / (P + R)
    SHD       = number of edge insertions/deletions/flips to match ground truth
    """
    n = direction.shape[0]
    tp = fp = fn = shd = 0

    for i in range(n):
        for j in range(n):
            pred = direction[i, j]
            true = ground_truth[i, j]
            if pred == 1 and true == 1:
                tp += 1
            elif pred == 1 and true == 0:
                fp += 1
            elif pred == 0 and true == 1:
                fn += 1
            if pred != true:
                shd += 1
    precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
```

```
recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
f1 = (2 * precision * recall / (precision + recall)
      if (precision + recall) > 0 else 0.0)
```

```
report = {
  "precision": round(precision, 3),
  "recall": round(recall, 3),
  "f1_score": round(f1, 3),
  "SHD": shd,
  "true_positives": tp,
  "false_positives": fp,
  "false_negatives": fn,
}
return report
```

```
gt_matrix = load_ground_truth("tep_ground_truth.csv", var_names)
report = evaluate_against_ground_truth(direction, gt_matrix)
print(report)
```


Step 11 — Visualize the discovered graph

```
import networkx as nx
import matplotlib.pyplot as plt

def visualize_graph(direction, var_names):
    G = nx.DiGraph()
    G.add_nodes_from(var_names)
    n = direction.shape[0]
    for i in range(n):
        for j in range(n):
            if direction[i, j] == 1:
                G.add_edge(var_names[i], var_names[j])

    plt.figure(figsize=(10, 8))
    pos = nx.spring_layout(G, seed=42)
    nx.draw(G, pos, with_labels=True, node_color="#a3c9f9",
            node_size=1400, font_size=7, arrowsize=15)
    plt.title("PC Algorithm - Discovered Causal Graph (Tennessee Eastman)")
    plt.tight_layout()
    plt.savefig("discovered_causal_graph.png", dpi=200)
    plt.show()

visualize_graph(direction, var_names)
```

Pipeline summary

load_dataset()	-> raw data + variable names
standardize()	-> z-scored data
partial_corr_test()	-> CI test (called repeatedly)
get_adjacent()	-> neighbor lookup helper
skeleton_discovery()	-> undirected skeleton + sepsets
orient_v_structures()	-> collider orientation
apply_meek_rules()	-> propagate remaining orientations
pc_algorithm()	-> orchestrates all of the above
evaluate_against_ground_truth()	-> precision / recall / F1 / SHD
visualize_graph()	-> plot final CPDAG

Reminder: `skeleton_discovery()` is the expensive step — its cost grows combinatorially with node degree via the `combinations(neighbors, ell)` loop, so it only stays tractable when the true plant graph is sparse (which TEP largely is, at ~32 edges over 33 variables).